



Patching code

Sometimes, we need to customize the way the UI works. Many common needs are covered by some supported API. For example, all registries are good extension points: the field registry allows adding/removing specialized field components, or the main component registry allows adding components that should be displayed all the time.

However, there are situations for which it is not sufficient. In those cases, we may need to modify an object or a class in place. To achieve that, Odoo provides the utility function `patch`. It is mostly useful to override/update the behavior of some other component/piece of code that one does not control.

Description

The `patch` function is located in `@web/core/utils/patch`:

`patch(objToPatch, extension)`

Arguments:

objToPatch (`object()`) – the object that should be patched

extension (`object()`) – an object mapping each key to an extension

Returns:

a function to remove the patch

The `patch` function modifies in place the `objToPatch` object (or class) and applies all key/value described in the `extension` object. An `unpatch` function is returned, so it can be used to remove the patch later if necessary.

Most patch operations provide access to the parent value by using the native `super` keyword (see below in the examples).

Patching a simple object

Here is a simple example of how an object can be patched:

```
import { patch } from "@web/core/utils/patch";

const object = {
```

```
};

patch(object, {
  fn() {
    // do things
  },
});
```

When patching functions, we usually want to be able to access the `parent` function. To do so, we can simply use the native `super` keyword:

```
patch(object, {
  fn() {
    super.fn(...arguments);
    // do other things
  },
});
```

Warning

`super` can only be used in a method not a function. This means that the following constructs are invalid for javascript.

```
const obj = {
  a: function () {
    // Throws: "Uncaught SyntaxError: 'super' keyword unexpected here"
    super.a();
  },
  b: () => {
    // Throws: "Uncaught SyntaxError: 'super' keyword unexpected here"
    super.b();
  },
};
```

Getters and setters are supported too:

```
    },  
    set number(value) {  
        super.number = value;  
    },  
});
```

Patching a javascript class

The `patch` function is designed to work with anything: object or ES6 class.

However, since javascript classes work with the prototypal inheritance, when one wishes to patch a standard method from a class, then we actually need to patch the `prototype` :

```
class MyClass {  
    static myStaticFn() {...}  
    myPrototypeFn() {...}  
}  
  
// this will patch static properties!!!  
patch(MyClass, {  
    myStaticFn() {...},  
});  
  
// this is probably the usual case: patching a class method  
patch(MyClass.prototype, {  
    myPrototypeFn() {...},  
});
```

Also, Javascript handles the constructor in a special native way which makes it impossible to be patched. The only workaround is to call a method in the original constructor and patch that method instead:

```
class MyClass {  
    constructor() {  
        this.setup();  
    }  
    setup() {  
        this.number = 1;  
    }  
}
```

```
setup() {  
    super.setup(...arguments);  
    this.doubleNumber = this.number * 2;  
},  
});
```

Warning

It is impossible to patch directly the `constructor` of a class!

Patching a component

Components are defined by javascript classes, so all the information above still holds. For these reasons, Owl components should use the `setup` method, so they can easily be patched as well (see the section on [best practices](#)).

```
patch(MyComponent.prototype, {  
    setup() {  
        useMyHook();  
    },  
});
```

Removing a patch

The `patch` function returns its counterpart. This is mostly useful for testing purposes, when we patch something at the beginning of a test, and unpatch it at the end.

```
const unpatch = patch(object, { ... });  
// test stuff here  
unpatch();
```

Applying the same patch to multiple objects

It could happen that one wants to apply the same patch to multiple objects but because of the way the `super` keyword works, the `extension` can only be used for patching once and

```
const obj1 = {
  method() {
    doSomething();
  },
};

const obj2 = {
  method() {
    doThings();
  },
};

function createExtensionObj() {
  return {
    method() {
      super.method();
      doCommonThings();
    },
  };
}

patch(obj1, createExtensionObj());
patch(obj2, createExtensionObj());
```

Warning

If an `extension` is based on another then the two extensions should be applied separately. Do not copy/clone an extension.

```
const object = {
  method1() {
    doSomething();
  },
  method2() {
    doAnotherThing();
  },
};

const ext1 = {
  method1() {
    super.method1();
  },
};
```

```
const invalid_ext2 = {
  ...ext1, // this will not work: super will not refer to the correct object in
  method2() {
    super.method2();
    doOtherThings();
  },
};

patch(object, invalid_ext2);
object.method1(); // throws: Uncaught TypeError: (intermediate value).method1 is

const valid_ext2 = {
  method2() {
    super.method2();
    doOtherThings();
  },
};

patch(object, ext1); // first patch base extension
patch(object, valid_ext2); // then the new one
object.method1(); // works as expected
```

Get Help

[Contact Support](#)[Ask the Odoo Community](#)